JavaScript

Fetch API & Async/Await

Detailed Lesson Guide

Course	JavaScript — Fetch API & Async/Await
Session	Session 1 of 1 · Introductory
Duration	30 Minutes
Depth	Beginner
Platform	Planrs Academy (Web + App)
Prepared by	Curriculum Design Team, Planrs Academy
Version	1.0 · May 2026

"Every child in India deserves to learn technology in a language and context that feels like home."
— Planrs Academy Curriculum Team

Contents

1. Learning Objectives

■	PRIMARY	Explain in their own words what it means to "fetch" data from the internet.
■	PRIMARY	Understand that some JavaScript actions take time (async) — the computer does not freeze while waiting.
■	PRIMARY	Read and identify the three keywords: async, await, and fetch().
■	PRIMARY	Run a provided code snippet and see real data appear on their webpage.
■	SECONDARY	Understand that the internet returns data as JSON — "a neatly packed dabba of information."
■	SECONDARY	Explain why we write async before a function name when it contains await.
■	STRETCH	Modify the provided code to display a different piece of information from the fetched data.

2. Key Vocabulary — The Three Magic Words

async

Marks a function that contains waiting steps. Must be written before the function keyword.

■■ Analogy ■ *Like the Bangalore traffic signal that tells you "this road has red-light stops ahead."*

```
// Example:
async function getData() { ... }
```

await

Pauses execution at that line until the promise resolves. Only valid inside an async function.

■■ Analogy ■ *Like placing your Zomato order — you pause your hunger-run to the kitchen and wait for the doorbell.*

```
// Example:
const response = await fetch(url);
```

fetch(url)

JavaScript's built-in function that sends a request to the given URL and returns a Response.

■■ Analogy ■ *Like sending the office peon with a letter to a government office — the peon goes, waits, and brings back the reply.*

```
// Example:
const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
```

.json()

A method on the Response object that reads the response body and parses it as JSON. Also needs await.

■■ Analogy ■ *Like opening the Mumbai Dabba — you call .json() to unpack the labelled compartments of data inside.*

```
// Example:
const data = await response.json();
```

3. Slide-by-Slide Lesson Walkthrough

SLIDE 1 Title Slide

Topic introduced: JavaScript — Fetch API & Async/Await

Hook: "Look at your favourite apps — where does all their information come from? Today you'll learn the superpower that answers that question!"

SLIDE 2 Where Does This Data Come From?

Engagement opener. Students look at familiar apps — games, weather, food delivery — and wonder aloud where the data lives.

Teacher tip: Ask students to name their favourite app and guess where its data comes from.

SLIDE 3 The Internet is Like a Chai Tapri

Analogy: You ask for "ek cutting chai, bhaiya" — the tapri-wala goes inside, makes it, brings it out. You didn't make it; you just asked and received.

The internet works exactly like this for data. Websites are the tapris; your browser is you placing the order.

SLIDE 4 APIs Are Like IRCTC Train Tickets

Technical definition (after analogy): An API is how programs ask each other for data.

fetch() is JavaScript's built-in tool to send those requests — like opening IRCTC to ask for train schedules without storing 12,000 trains on your phone.

SLIDE 5 Async Means "Don't Freeze, Just Wait"

Analogy: Order food on Zomato. You don't stand frozen at the door for 30 minutes. You watch TV, play, study. When the doorbell rings — you collect your food.

Code does the same: it places the "order" (fetch request), keeps doing other things, and comes back when data arrives.

SLIDE 6 Mumbai Local Train is Async Too!

"Train arrives in 3 minutes." You don't freeze. You check your phone, talk to a friend. When the train comes, you board it.

Async code = your computer keeps working while waiting for data. It is NEVER frozen.

SLIDE 7 fetch() is Like Sending a Peon

Analogy: You send the office peon with a request letter to a government office. The peon waits in the queue, gets the document, brings it back.

Code introduced:

```
async function getData() {
let response = await fetch('https://api.example.com/data');
}
```

SLIDE 8 async / await Are Like Traffic Signals

async = marks your function: "this function has waiting steps inside."

await = pauses at that exact line, like a red signal, until the green comes (data arrives). Then it moves on.

Bangalore traffic signal analogy makes this concrete and culturally familiar.

SLIDE 9 JSON — The Neatly Packed Mumbai Dabba

Analogy: JSON is like a tiffin dabba with labelled compartments — name here, city there, age in another box.

We call .json() to open the dabba and read what's inside.

```
// The internet sends back something like this:
{ "name": "Aarav", "city": "Pune", "age": 9 }
```

SLIDE 10 Live Code Demo — Magic Data Fetcher

The basic fetch pattern shown and explained line by line. Students see the three keywords in real code for the first time.

Key point: fetch() asks the internet, await says "wait here patiently," .json() unpacks the data.

SLIDE 11 Live Code Demo — Full HTML (Part 2)

Complete HTML page shown: a button with onclick calling getData(), and a that receives innerHTML.

Teacher tip: Type this live. Let students watch the page come together in real time.

SLIDE 12 Live Code Demo — Result on Screen

"Look! The Name and City now show on our webpage. We fetched REAL data from the internet!"

"Shabash! You did it! ■" — Students see the payoff moment. This is the delight milestone of the session.

SLIDE 13 Displaying Data with innerHTML

Connects to prior knowledge: students already know getElementById() and innerHTML from DOM lessons.

"It's like writing on a chalkboard for everyone to see — you're putting the fetched data on the board so your whole page can read it!"

SLIDE 14

Quick Quiz Time!

Energy moment: "Raise your hand if you're excited!" — re-engages learners before assessment.

Five quick questions follow across the next two slides (see Section 5 for full Q&A;).

SLIDE 15

Quiz — Part 1

Q1: What does fetch() do? → Asks the internet for data.

Q2: What is async/await like? → Ordering food and waiting for delivery — without freezing!

SLIDE 16

Quiz — Part 2

Q3: What keyword goes before a function with await? → async

Q4: What does await do? → Pauses and waits for data without freezing.

Q5: What is JSON like? → A neatly packed tiffin dabba!

SLIDE 17

Wrap-Up & Homework

Recap: students can now explain fetch(), async/await, and JSON using Indian everyday examples.

Homework task: Open a text editor, type the code from the live demo, change /users/1 to /users/2 or /users/3, and see different data appear!

Next session preview: "What happens if the internet is slow or the request fails? We'll learn how to handle that!"

SLIDE 18

Thank You!

"Keep exploring the magic of JavaScript!" — Planrs Academy

4. Full Code Example (Copy & Try!)

This is the complete, working code from the lesson demo. Students type or paste this into a .html file and open it in a browser.

```
<!-- Save this as: my-first-fetch.html -->
<!DOCTYPE html>
<html>
<body>

<h2>My Data Fetcher ■</h2>
<button onclick="getData()">Click to Fetch!</button>
<div id="result"></div>

<script>

// async = "this function has waiting steps inside"
async function getData() {

// await = "pause here until the data arrives"
// fetch() = "send the peon to get information from this address"
const response = await fetch(
'https://jsonplaceholder.typicode.com/users/1'
);

// .json() = "open the dabba and read what's inside"
const data = await response.json();

// innerHTML = "write this on our page for everyone to see"
document.getElementById('result').innerHTML =
'Name: ' + data.name + '<br>City: ' + data.address.city;
}

</script>
</body>
</html>
```

What you will see on screen:

```
Name: Leanne Graham City: Gwenborough
```

■ Stretch Challenge: Change `data.name` to `data.email` — what shows on screen now?

5. Quick Quiz — Questions & Answers

Q1 **What does fetch() do in JavaScript?** ■ Recall

✔ Answer: It asks the internet for data from a given web address (URL) and brings back a response — just like sending the office peon with a request letter!

Q2 **What is async/await similar to in real life?** ■ Recall

✔ Answer: Ordering food on Zomato — you place the order (fetch), then keep doing other things (async), and collect the food when the doorbell rings (await resolves).

Q3 **What keyword must you write before a function name when it contains an await inside?** ■ Understanding

✔ Answer: async — it marks the whole function as "this function has waiting steps, so JavaScript knows to handle it specially."

Q4 **What does await do when the JavaScript engine reaches it?** ■ Understanding

✔ Answer: It pauses execution at that exact line, like a red traffic signal, and resumes only when the data (promise) has arrived. The rest of the page does NOT freeze.

Q5 **What is JSON and how do we "open" it in JavaScript?** ■ Application

✔ Answer: JSON is a neatly packed format for data — like a Mumbai tiffin dabba with labelled compartments. We call .json() on the response to open the dabba and read the values inside.

6. Drag & Drop Activity Guide

The interactive HTML activity ("Enchanted Forest") contains three drag-and-drop spells that reinforce the three key skills. Use this guide alongside the activity.

■ Spell 1: Build the Fetch Call

Goal	Students drag async, await (×2), fetch, and json() into the correct positions in a real function.
Decoy chips	return, console, and a second await are included as decoys to require genuine understanding.
Drop zones	5 drop zones: (1) async before function, (2) await before fetch, (3) fetch keyword, (4) await before .json(), (5) json.
Success	All 5 filled correctly → Spell 1 popup unlocks with the Zomato analogy recap.

■ Spell 2: Async Flow with Spinner

Goal	Students drag 4 steps in order: async, showSpinner(), await, hideSpinner() — demonstrating understanding of sequence.
Live feedback	The spinner widget actually animates when showSpinner() is dropped, then stops when hideSpinner() is dropped.
Decoy chips	catchError() and console.log() are present as distractors.
Success	Correct order → Spell 2 popup with traffic-signal analogy recap.

■ Spell 3: Display the Result on Screen

Goal	Students complete getElementById("joke-box").innerHTML = data.joke; by dragging the 4 missing pieces.
Live feedback	A sample joke text appears in the result box the moment all 4 spots are filled correctly.
Decoy chips	src, value, and body are included as wrong-answer options.
Success	Correct → Final popup with keyword pills (async · await · fetch) and trophy celebration.

7. Homework & Next Session Preview

■ Homework Task

1. Open Notepad (or any text editor).
2. Type the full code from Section 4 of this guide.
3. Save the file as my-first-fetch.html.
4. Open the file in a web browser and click the button.
5. Change `/users/1` to `/users/2` — what different name and city appears?
6. Stretch: Change `data.name` to `data.email` and see what happens!

■ Next Session Preview

In the next session we will answer: **"What happens if the internet is slow or the request fails?"**
We will learn try/catch — the safety net for async code — and build more robust fetch programs.

async	**await**	**fetch(url)**	**.json()**
marks a function with waiting steps	pauses at this line until data arrives	sends the request to the internet	unpacks the data dabba

"If a child who has never heard the word API can explain fetch() using a real-life Indian example and see data appear on screen — the session has succeeded." — Planrs Academy Quality Gate

8. Practice Questions for Students

Test your understanding of JavaScript Fetch API, Async/Await, and JSON

> **Instructions:** Complete the following questions individually. Read each code snippet carefully before answering. (Answers are omitted for student self-assessment / classroom test).

Question 1 (Multiple Choice)

What is the main purpose of the `fetch()` function in JavaScript?

A) To freeze the browser until data loads

B) To send a HTTP request to a server URL to retrieve data

C) To style HTML elements with CSS

D) To create a loop that repeats 10 times

Question 2 (Fill in the Blanks)

Fill in the missing keywords in the code snippet below to correctly declare an asynchronous function and wait for the network response:

```
_____ function loadUserData() {
    const response = _____ fetch('https://jsonplaceholder.typicode.com/users/1');
    const data = _____ response.json();
    console.log(data);
}
```

Question 3 (Real-World Analogy)

Explain in 2-3 sentences why making an asynchronous API request with `async/await` is like ordering food on Zomato rather than standing frozen at the door.

Question 4 (Multiple Choice)

Why must we call `await response.json()` after calling `fetch()`?

A) To convert the raw HTTP response body into a usable JavaScript object

B) To restart the user's web browser

C) To save the data permanently into a database

D) To encrypt the password before sending it

Question 5 (Code Analysis & Output)

Given the JSON response below returned from an API:

```
{
  "name": "Aarav Sharma",
  "email": "aarav@example.com",
  "address": {
    "city": "Pune",
    "zipcode": "411001"
  }
}
```

What will be printed on the screen if the code executes:
`document.getElementById('result').innerHTML = data.name + " from " + data.address.city;?`
